LLM-Powered Data Processing

From Chat to Batch: LLMs as Data Operators

Bryan Bischof

8 Apr 2026

Today's Journey

Units 9 and 10 shifted who owns the work—from a person in a chat thread to **agents** with harnesses, subagents, and verifier loops.

This unit is a third setting: the client is a **pipeline** (batch jobs, SLAs, millions of records). Agent-style loops still appear inside—what changes is who owns the loop.

The Through-Line

LLMs went from answering questions to doing work—and that required solving increasingly hard engineering problems.

Five Key Ideas:

1. LLMs as data operators
2. The accuracy problem
3. The economics problem
4. Eval as the scaling lever
5. The trust problem

For each, we ask P^3:

- What's the **promise**?
- How do you **prove** it?
- What makes it **production**-grade?

Today's Session

LLMs as **data operators** at scale — primitives, batch failure modes, case studies (trust, economics, eval, provenance), declarative pipelines (MCP, skills), P^3 throughout.

Same lens all day

For every idea: **Promise** → **Proof** → **Production** (P^3).

DocETL and DocWrangler (Context)

Two systems from UC Berkeley's EPIC Lab that you will see in depth today.

DocETL [1]: declarative pipeline framework with an **optimizer**—write LLM operators (map, reduce, resolve. . .) and the system rewrites the pipeline for better accuracy: inserting decomposition steps, adding **gleaning loops** (generate → validate → re-prompt), detecting when resolve should precede reduce.

DocWrangler [18]: interactive layer on top—inspect outputs per step, annotate errors, have the system **refine prompts** from your annotations. Analyze–Measure–Improve applied to pipeline dev, not prompt engineering.

Both reappear in the **Declarative Pipelines** section later in this deck.

After Units 9 and 10

Unit 9: agency and the **harness** — policy, state, verification, subagents, trajectory-first proof.

Unit 10: decomposition — isolation vs shared context, fan-out / fan-in, verifier-style loops, planner–executor handoffs.

This unit: the consumer is often a **pipeline** (batch, SLA, millions of records) — same mechanisms, different non-functional requirements.

- Harness → schemas, validators, checkpoints, provenance traces
- Verifier pattern → extract-then-validate at record scale
- Scoped stages → Harvey-like pipelines (handoffs, not one giant prompt)
- Fan-out / gather → parallel map + reduce (context limits, ordering)

Outline

1. Warm-up and Analytical Lens
2. Key Idea 1: LLMs as Data Operators
3. Key Idea 2: The Accuracy Problem
4. Key Idea 3: The Economics Problem
5. Key Idea 4: Eval as the Scaling Lever
6. Key Idea 5: The Trust Problem
7. Putting It All Together

When Chat Isn't the Product

Consider what these companies actually need:

- **Walmart** needs to extract color, size, and material from product descriptions—across **850 million** catalog entries [9]
- **Elysian** (commercial insurance automation) needs to review insurance claims averaging **5,400 pages** of scanned faxes and handwritten annotations [8]
- **Abridge** (clinical AI documentation) needs to generate clinical notes from doctor-patient conversations—where a single hallucinated detail could harm a patient [4]

None of these are chat problems. They're **data-processing problems** that happen to require language understanding.

The LLM isn't the product—it's an **operator** *inside a larger system.*

The Recurring Question (Our Analytical Lens)

For each key idea, we'll ask the same three questions—mapping directly to P^3:

1. **What's the promise?** What does the system need to deliver, and what are the acceptance criteria?
2. **How do you prove it?** How did the team measure quality before scaling—and how do they measure it continuously?
3. **What makes it production-grade?** What engineering decisions keep the proof valid at scale?

P^3 lens

Promise → Proof → Production. Each section introduces a key idea, shows how real companies solved it, and connects it back to this framework.

Outline

1. Warm-up and Analytical Lens
2. Key Idea 1: LLMs as Data Operators
3. Key Idea 2: The Accuracy Problem
4. Key Idea 3: The Economics Problem
5. Key Idea 4: Eval as the Scaling Lever
6. Key Idea 5: The Trust Problem
7. Putting It All Together

The Conceptual Shift

Before LLMs: data processing meant rules and trained classifiers—regex for extraction, supervised models for classification, templates for generation. Each task needed its own labeled dataset, its own model, its own deployment.

After LLMs: a single model can classify, extract, summarize, resolve entities, and enrich records—guided by a prompt instead of a training set.

Adding a new category or handling a new document format?

Before: retrain. **After:** re-prompt.

The Fundamental Primitives

Operator	What it does	SQL analogy
Map	Transform each record independently	`SELECT f(col)`
Filter	Keep or discard records	`WHERE`
Classify	Assign a label from a fixed set	`CASE WHEN`
Reduce	Aggregate multiple records into one	`GROUP BY + AGG`
Resolve	Deduplicate / merge same-entity records	Self-join + merge
Enrich	Add new fields from existing content	Computed column

Every operator produces **structured output**—a Pydantic model or JSON schema that the next step can parse and validate. This is the interface contract between pipeline steps.

Structured Output at Batch Scale

In a chat, a natural-language response is fine. In a pipeline, you need **structured output**—typed fields, constrained decoding, schema enforcement—so the next step can parse, validate, and route.

At batch scale, this matters *even more* than in chat:

- A 1% parse-failure rate on 50K records = **500 broken records**
- Every operator should declare its output schema explicitly
- Schema enforcement (constrained decoding from Unit 5) eliminates an entire class of runtime errors
- The extract-then-validate pattern (Key Idea 2) depends on having structured output from step 1

Under the hood, every operator is the function-calling mechanism from Unit 5: structured input → model call with schema → typed output. The operator abstraction is syntactic sugar over that loop.

The Classify Operator

A classify step for support tickets—structured input → schema-enforced call → typed output:

classify_ticket

```
from openai import OpenAI
from pydantic import BaseModel
from typing import Literal

class TicketClassification(BaseModel):
    category: Literal[
        "billing", "technical", "account", "security", "other"
    ]
    priority: Literal["low", "medium", "high"]
    confidence: float

client = OpenAI()

def classify_ticket(ticket: dict) -> TicketClassification:
    return client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        response_format=TicketClassification,
        messages=[
            {"role": "system", "content": CLASSIFY_PROMPT},
            {"role": "user", "content":
                f"Subject: {ticket['subject']} Body: {ticket['body']}"},
        ],
    ).choices[0].message.parsed
```

Every operator—map, filter, classify, enrich—follows this same pattern.

The MapReduce Analogy

If you've seen MapReduce in a distributed systems context, the parallel is direct—and the differences are instructive.

Classic MapReduce:

- **Map** applies a deterministic function to each record independently. Same input → same output, every time.
- **Reduce** aggregates using an associative, commutative combiner. `sum(a, b) = sum(b, a)`—order doesn't matter.

LLM MapReduce:

- **Map** applies a *prompt* to each record. Same input → *similar* output most of the time, but nondeterministic.
- **Reduce** asks the LLM to summarize across records. Order *does* matter, context limits constrain, output is lossy.

Classic vs. LLM MapReduce

	Classic MapReduce	**LLM MapReduce**
Map function	Deterministic	Nondeterministic (prompt-based)
Reduce function	Associative, commutative	Order-sensitive, context-limited
Failure mode	Crash (retry works)	Silent wrong answer (retry $\rightarrow$ *different* wrong answer)
Scaling	Add machines	Add API concurrency (rate-limited)
Cost driver	Compute time	Tokens $\times$ price per token

Parallelism carries over—map is still embarrassingly parallel. Fault tolerance doesn't carry over—retrying a failed LLM call may produce a different (not necessarily better) result. And reduce needs careful architectural attention.

Fan-Out / Fan-In: Runtime Parallelism

In LLM pipelines, the number of parallel tasks often depends on the *input data itself*—a document with 3 sections needs 3 extractors, another might need 50.

This is the **fan-out / fan-in** (scatter-gather) pattern:

```
Input → Split into N chunks (N depends on document)
        ↳ LLM call (chunk 1)
        ↳ LLM call (chunk 2)
        ↳ ...
        ↳ LLM call (chunk N)
            ↳ Gather / reduce results
```

Each parallel branch can carry its own state (different chunk text, different schema fields, different model choice) while sharing a common output format for the gather step.

This is how you process a 200-page contract: split into sections, fan out extraction across all sections in parallel, fan in the structured results.

Checkpointing: Human-in-the-Loop at the Record Level

In batch processing, some records need human review. Start with a **binary** gate: every record either **passes** or gets **flagged**.

1. Process each record through the pipeline
2. If a validation step fails (schema error, missing field, disallowed label) or the output needs human feedback, **flag** the record and save its state
3. Continue processing the rest of the batch
4. A human reviews flagged records and provides corrections
5. Corrected records re-enter the pipeline from where they stopped

Once binary works, graduate to **thresholds**—confidence scores, redundant-call agreement, domain rules—that route records into pass / review / reject buckets.

Either way, the batch doesn't block on human review. Each record maintains its own checkpoint—resuming one doesn't affect others.

Map: The Workhorse

Most pipeline steps are **map** operations—apply the same prompt to each record. Parallelism is straightforward:

llm_map—parallel LLM map with semaphore

```
async def llm_map(
    records: list[dict],
    prompt_template: str,
    output_schema: type[BaseModel],
    model: str = "gpt-4o-mini",
    concurrency: int = 20,
) -> list[BaseModel]:
    """Apply an LLM prompt to each record in parallel."""
    semaphore = asyncio.Semaphore(concurrency)

    async def process_one(record: dict) -> BaseModel:
        async with semaphore:
            prompt = prompt_template.format(**record)
            return await call_llm(prompt, output_schema, model)

    return await asyncio.gather(
        *[process_one(r) for r in records]
    )
```

Reduce: Where the Analogy Breaks

LLM reduce is hard because:

- **Context limits:** 100 records $\times$ 500 tokens each $=$ 50K tokens. You can't always fit everything in one call.
- **Order effects:** which records the LLM sees first influences the output (recall lost-in-the-middle from Unit 2).
- **No commutativity:** $\texttt{reduce}(A, B) \neq \texttt{reduce}(B, A)$ in practice.

LLM reduce isn't just "call the model on a bigger input." It requires choosing a strategy, deciding what information to preserve at each level, and often pre-processing with map to make the reduce tractable.

Reduce Strategies

Three strategies, each with tradeoffs:

Strategy	How it works	Accuracy	Cost
Stuff	Fit all records into one call	Best (if it fits)	One call
Incremental (fold)	Process one at a time, updating running summary	Recency-biased	N calls
Tree reduce	Reduce pairs $\rightarrow$ reduce results $\rightarrow \ldots$ until one remains	Balanced	$\log(N)$ calls

The right choice depends on how many records you're reducing and whether the full set fits in context. DocETL's optimizer can detect when stuff-reduce fails and automatically switch to tree reduce.

Worked Example: Patient Discharge Summary

A patient sees three specialists. Each produces a clinical note (∼800 tokens). The discharge summary must integrate all three with no contradictions.

```
Cardiology note   ↪
Neurology note    ↪ LLM reduce → Integrated discharge summary
Radiology report  ↪
```

This is a **stuff** reduce—three notes fit in context.

But the task is harder than it sounds:

- Reconcile overlapping medication lists
- Resolve conflicting observations ("stable" vs. "concerning")
- Preserve every clinician's key findings without hallucinating connections

A traditional `GROUP BY` would never face this problem.

Worked Example: 200 Customer Interviews

A product team has 200 interview transcripts (∼2,000 tokens each = 400K total). They want 5–7 themes with supporting quotes.

```
Interviews (200)
  ↪ Map: extract key quotes + preliminary themes per interview
  ↪ Tree reduce (depth 3): merge 200 → 25 → 4 → 1
  ↪ Final summary: 5-7 themes with representative quotes
```

Why not stuff?
400K tokens—impossible.

Why not fold?
Recency-biased—later interviews overwrite earlier themes.

Why tree reduce works:
Merge clusters of ∼8 interviews into theme summaries, then merge those summaries. Each merge level is a "summarize and integrate" call.

Information is lost at every level—the art is losing the *right* information.

Worked Example: Financial Trend Analysis

An analyst wants a trend summary across 50 10-Q filings for one company.

```
10-Q filings (50)
  ↪ Map: extract {revenue, margin, risk_factors} per quarter
  ↪ Sort by date (deterministic, not LLM)
  ↪ Reduce: "Given these 50 quarterly snapshots, identify
             trends, inflection points, and emerging risks"
```

Here the **map** step produces structured data, and the **reduce** operates on structured intermediate results—not raw text.

Common pattern

Use map to compress unstructured documents into structured summaries, then reduce over the structured output. The reduce input is much smaller and more predictable.

The Cold-Start Advantage

DoorDash uses LLMs for attribute extraction across their merchant catalog [13]. New merchants onboard constantly, each with different menu formats and zero labeled data. Traditional NLP needs labeled examples per category—retraining for every new merchant. LLMs generalize from instructions.

Instacart's PARSE system uses multimodal LLMs to extract product attributes (flavor, size, nutritional content) from millions of SKUs [11]. It handles ambiguities that rule-based and traditional ML systems couldn't—like distinguishing a product's primary flavor from variant flavors in the same description.

P^3 lens — Promise

"Extract structured attributes from any merchant's data without task-specific training." Acceptance criteria: coverage (what % of SKUs have correct attributes) and accuracy (what % of extracted values are right).

Outline

1. Warm-up and Analytical Lens
2. Key Idea 1: LLMs as Data Operators
3. Key Idea 2: The Accuracy Problem
4. Key Idea 3: The Economics Problem
5. Key Idea 4: Eval as the Scaling Lever
6. Key Idea 5: The Trust Problem
7. Putting It All Together

Errors Compound

If each step in your pipeline has 95% accuracy, a 5-step pipeline has:

$$0.95^5 = 0.774$$

Nearly **1 in 4 records** has at least one error.

In a chat:
The user re-asks. Self-correcting.

In a pipeline:
Bad data propagates silently into downstream systems. No one re-asks.

The central engineering challenge

This compounding error rate is the reason every subsequent key idea exists. Accuracy, economics, eval, and trust are all responses to this fundamental problem.

Pattern: Extract-Then-Validate

The most widespread solution is a two-agent pattern: one LLM does the work, a second LLM (or deterministic check) validates it.

Walmart [9]:

- One LLM extracts product attributes
- A separate LLM quality-checks the extraction
- Result: improved over **850 million** product data entries
- Superior recall vs. supervised models on diverse descriptions

Two agents—one to work, one to check—is the simplest reliable architecture.

Harvey AI: Agentic Multi-Step Extraction

Harvey AI takes extract-then-validate further with agentic multi-step extraction for legal documents [5]:

- Each sub-agent handles a specific reasoning task: clause identification, cross-reference resolution, risk classification
- Sub-agents follow the subagent pattern from Unit 5: each with its own context, tools, and system prompt

Result

98.47% accuracy on SPA deal-point extraction, vs. **66%** for raw GPT-4o on the same documents [5]. The model is the same—the **pipeline architecture** makes the difference.

P^3 lens — Proof

Harvey built **BigLaw Bench**, a custom benchmark developed with law firms to measure real legal task performance [6]. They defined the bar first, measured against it, and published the results.

Harvey: The Subagent Architecture

Each subagent gets its own system prompt, tool set, and model choice:

Parent orchestrator for SPA deal-point extraction

```
async def analyze_spa(document: str) -> SPAAnalysis:
    clauses = await run_subagent(
        system_prompt=CLAUSE_ID_PROMPT,
        tools=[search_document, list_sections],
        model="gpt-4o-mini",
        input=document,
        output_schema=list[Clause],
    )
    cross_refs = await run_subagent(
        system_prompt=CROSS_REF_PROMPT,
        tools=[lookup_defined_term, resolve_reference],
        model="gpt-4o",           # harder task, better model
        input=clauses,
        output_schema=list[ResolvedClause],
    )
    risk = await run_subagent(
        system_prompt=RISK_CLASSIFY_PROMPT,
        tools=[],                  # pure classification, no tools
        model="gpt-4o-mini",
        input=cross_refs,
        output_schema=list[RiskAssessment],
    )
    return SPAAnalysis(
        clauses=clauses,
        cross_references=cross_refs,
        risk_assessments=risk,
    )
```

Pattern: Decomposition

When documents are too long or tasks too complex for a single call:

```
Split → Map → Gather
```

1. **Split** the document into chunks (by section, page, paragraph)
2. **Map** the LLM operation over each chunk independently
3. **Gather** context from surrounding chunks for local context

Elysian + Reducto [8] faces this at the extreme—5,400-page insurance claims with scanned faxes and handwritten annotations. Reducto's pipeline runs three stages:

1. Computer vision layout parsing to segment pages into regions
2. Vision-language models to interpret each region
3. Agentic OCR with multi-pass correction until confidence gates pass

Result: **16x faster** claim review.

Before you can run LLMs on documents, you need reliable **ingestion**. The parsing pipeline is often harder than the analysis pipeline.

Validation Between Steps: Mixing LLM and Deterministic Tools

A realistic pipeline interleaves LLM operators with deterministic tool calls—the LLM handles judgment, the tools handle precision:

Product enrichment: judgment + precision

```
async def enrich_product(product: dict) -> EnrichedProduct:
    barcode_data = await barcode_api.lookup(product["upc"])
    pricing = await pricing_db.get(product["sku"])

    attributes = await call_llm(
        prompt=EXTRACT_ATTRS_PROMPT.format(
            description=product["description"],
            image_url=product.get("image_url"),
        ),
        output_schema=ProductAttributes, model="gpt-4o-mini",
    )
    summary = await call_llm(
        prompt=SUMMARIZE_PROMPT.format(
            name=product["name"],
            attributes=attributes.model_dump_json(),
            price=pricing.current_price,
        ),
        output_schema=ProductSummary, model="gpt-4o-mini",
    )
    # Deterministic validation -- no LLM needed
    taxonomy_check = taxonomy_service.validate(
        category=attributes.category, subcategory=attributes.subcategory,
    )
    return EnrichedProduct(**product, barcode=barcode_data, pricing=pricing,
        attributes=attributes, summary=summary,
        taxonomy_valid=taxonomy_check.is_valid)
```

What to Validate Between Steps

Every step can fail silently. Insert validation between LLM steps:

- **Schema validation:** does the output parse? Required fields present?
- **Invariant checks:** labels from the allowed set? Dates in range?
- **Confidence filtering:** route uncertain outputs to a more expensive model or human review
- **Cross-record consistency:** do extracted entities match across related documents?

Validation tools are cheap, deterministic, and catch obvious errors before they propagate. They're the "immune system" of the pipeline—most of the time they do nothing, but when they fire they prevent cascading failures.

Outline

1. Warm-up and Analytical Lens
2. Key Idea 1: LLMs as Data Operators
3. Key Idea 2: The Accuracy Problem
4. Key Idea 3: The Economics Problem
5. Key Idea 4: Eval as the Scaling Lever
6. Key Idea 5: The Trust Problem
7. Putting It All Together

Cost at Volume

At scale, token costs dominate:

Scenario	Model	Cost (50K records)
800 in / 200 out tokens per record	gpt-4o-mini	~$12
Same tokens	gpt-4o	~$600
gpt-4o with 5-step pipeline	gpt-4o × 5	~$3,000

Model choice matters enormously at volume. A 50x cost difference between gpt-4o-mini and gpt-4o means the difference between "cheap experiment" and "needs VP approval."

Pattern: Model Routing

Not every record needs the same model. Route cheap-model-first, escalate on uncertainty:

```
Record → Cheap Model → Confidence check
                        |
              High → Accept
              Low  → Expensive Model → Accept
```

The routing decision is part of the architecture, not an afterthought. Most records are easy—the cheap model handles them. The expensive model only sees the hard cases.

Amazon: Self-Learning Routing

Amazon's catalog team built a self-learning version of model routing [10]:

- Multiple smaller models process routine products
- When they **disagree**, a larger supervisor investigates
- Key innovation: disagreements generate **reusable learnings** stored in a dynamic knowledge base
- Over time, the cheap models handle increasingly complex cases *without needing the supervisor*

The system teaches itself. The knowledge base grows from production disagreements, making the cheap path wider and the expensive path narrower over time.

Unit 10 bridge: Same design space as routing and escalation in multi-agent systems—tuned here for catalog-scale throughput and spend.

Hebbia: Multi-Model Orchestration

Hebbia routes across o3-mini, o1, and GPT-4o depending on step complexity [7]:

- **92%** accuracy on complex financial tasks, vs. **68%** for traditional RAG
- Each analytical step is a subagent with its own model and tool set
- The multi-agent pattern from Unit 5, applied to document analysis rather than chat

Structured insights from 10K+ documents per query. The routing decision is what makes this economically viable—not every step needs the most expensive model.

Unit 10 bridge: Per-step model choice is specialization/routing—document analysis at scale rather than an interactive agent session.

Pattern: Fine-Tuning for Economics

Ramp processes every corporate receipt through an LLM: OCR → fine-tuned model → structured JSON [14].

1. Started with a general-purpose model to **prove the task works**
2. Built a golden dataset and measured accuracy
3. Fine-tuned a smaller model for production

Result:

- **97%** categorization accuracy
- **34%** fewer receipts needing manual review
- **79% cost savings** vs. the general-purpose API

Fine-tuning is a **cost optimization**, not a capability unlock. Prove the task first with an expensive model, then optimize.

P^3 Lens — Production (Ramp)

P^3 lens

Ramp's story is a textbook P^3 arc:

- **Promise:** categorize receipts accurately at scale
- **Proof:** general-purpose model demonstrates the task is feasible; golden dataset measures 97% accuracy
- **Production:** fine-tune for economics, batch-process on Modal with 256 parallel workers, monitor the 34% manual-review rate as an ongoing metric

The trajectory: general model → measure → fine-tune → 79% savings. Proof first, then optimize for production economics.

Batch APIs and Throughput

Provider batch endpoints change the math:

- **OpenAI Batch API:** 50% off, 24-hour turnaround
- **Anthropic Message Batches:** 50% off, up to 100K requests per batch

Instacart's Maple service goes further [12]—an internal batch platform that runs LLM workloads across teams (Catalog, Fulfillment, Search):

- **50% cost reduction** vs. real-time API calls
- Uses **Temporal** for fault tolerance
- Uses **Parquet** for data handling
- Shared infrastructure across teams

This is what the **Production** phase of P^3 looks like for LLM data processing at scale.

Outline

1. Warm-up and Analytical Lens
2. Key Idea 1: LLMs as Data Operators
3. Key Idea 2: The Accuracy Problem
4. Key Idea 3: The Economics Problem
5. Key Idea 4: Eval as the Scaling Lever
6. Key Idea 5: The Trust Problem
7. Putting It All Together

The Three-Layer Eval Strategy

You can't human-label 50,000 outputs. So how do you know if your pipeline works?

Layer	What it covers	Cost	Coverage
Invariant checks	Schema, label sets, ranges, non-null, cross-field consistency	Free	100%
Sampled human eval	Stratified sample (200–500), human-labeled, per-step metrics	Expensive	1–5%
LLM-as-judge	Subjective quality checks on a sample	Cheap	5–20%

Layer 1 is your safety net (free, 100% coverage). Layer 2 is ground truth (expensive, sampled). Layer 3 bridges the gap (cheap, broader).

Spotify's Eval Story

Spotify replaced an entire stack of specialized ML models (sentiment, topic modeling, speech classification, ad detection) with a single LLM for podcast previews [2]:

- **4.6%** engagement lift
- **5x** processing efficiency

But the **eval story** is the real contribution. They built a "profile-aware LLM-as-judge" [3]:

1. Summarize a listener's preferences from ∼90 days of activity
2. Have the LLM score how well candidate episodes match
3. **75%** alignment with listener judgments

Good enough to filter candidates without running an A/B test on everything.

P^3 Lens — Proof (Spotify)

P^3 lens — Proof

Spotify's judge is a **proof instrument** calibrated against real user behavior data. It sits between slow-but-accurate human labeling and fast-but-noisy offline metrics.

The feedback loop:

1. Judge scores → identify underperforming recommendations
2. Improve the pipeline
3. Re-score

The judge doesn't replace A/B testing—it reduces how much A/B testing you need by filtering out clearly bad candidates offline.

Progressive Processing

A practical pattern: **don't** process 50K records and then evaluate. Instead:

1. Process a **sample** (200–500 records)
2. Run full eval (human + invariants + judge)
3. Fix issues in the prompt / pipeline
4. Scale to full corpus
5. Run invariants on 100% and judge on a sample
6. Feed new failures back into the eval set

This is the Analyze–Measure–Improve loop from Unit 3, applied to batch processing. You iterate on a small sample until quality is right, then scale—and keep monitoring.

Outline

1. Warm-up and Analytical Lens
2. Key Idea 1: LLMs as Data Operators
3. Key Idea 2: The Accuracy Problem
4. Key Idea 3: The Economics Problem
5. Key Idea 4: Eval as the Scaling Lever
6. Key Idea 5: The Trust Problem
7. Putting It All Together

Provenance: Tracing Every Output to Its Source

In high-stakes domains, "the model said so" isn't enough. You need to show **where** in the source data each output came from.

Abridge [15]:

- Generates clinical notes from doctor-patient conversations
- Every sentence in the note links back to the source audio
- 1,000+ hours of human validation [4]
- 6x more likely to catch and fix hallucinations vs. off-the-shelf

Harvey [5]:

- Traces every extracted deal point back to the specific clause and cross-reference in the source SPA
- Provenance is architecturally required, not optional

In legal and medical contexts, an output you can't trace is an output you can't use.

Abridge: Confabulation Elimination

Published research on "confabulation elimination" [4] describes purpose-built guardrails:

- Provenance tracking as an **architectural choice**—every generated sentence traces to source audio
- 1,000+ hours of human validation backing the guardrails
- **6x** more likely to catch and fix hallucinations vs. off-the-shelf models

They've gone further with **specialty-specific models** [15]:

- Tuned for neurology (precise symptom chronology)
- Tuned for surgery (risk-benefit documentation)
- Deployed without requiring clinicians to change their workflow

P^3 lens — all three

Promise: "generate clinical notes without fabricating information." **Proof:** 1,000+ hours of human validation, confabulation detection research, specialty-specific evaluation [4, 16]. **Production:** provenance tracking, specialty models behind the same interface, continuous monitoring through structured evaluation checklists.

Outline

1. Warm-up and Analytical Lens
2. Key Idea 1: LLMs as Data Operators
3. Key Idea 2: The Accuracy Problem
4. Key Idea 3: The Economics Problem
5. Key Idea 4: Eval as the Scaling Lever
6. Key Idea 5: The Trust Problem
7. Putting It All Together

P^3 Matrix Across Case Studies

Company	Promise	Proof	Production
Harvey	Extract deal points with lawyer-grade accuracy	BigLaw Bench (custom eval with law firms) [5, 6]	Agentic pipeline with sub-agents; Vault processes 1000s of docs per matter
Hebbia	Structured insights from 10K+ docs per query	92% accuracy vs. 68% RAG baseline [7]	Multi-model routing (o3-mini / o1 / GPT-4o); distributed orchestration
Walmart	Correct attributes across 850M catalog entries	Two-agent extract + validate loop [9]	LLMs replaced work that would have taken 100x manual labor
Amazon	Self-improving catalog extraction	Consensus-based routing; disagreements become learnings [10]	Dynamic knowledge base; smaller models improve over time
Instacart	Flavor, size, nutrition from millions of SKUs	PARSE multimodal extraction [11]	Maple batch service across teams; 50% cost reduction [12]
DoorDash	Attribute extraction for any new merchant	LLMs generalize from instructions [13]	Knowledge graph enrichment; continuous onboarding
Ramp	97% receipt categorization at scale	General model $\rightarrow$ measure $\rightarrow$ fine-tune $\rightarrow$ remeasure [14]	Fine-tuned model on Modal; 256 parallel workers; 79% cost savings
Spotify	Better podcast previews than specialized ML stack	Profile-aware LLM-as-judge (75% listener agreement) [2, 3]	Replaced 5 specialized models; 5x processing efficiency
Abridge	Clinical notes without fabrication	1,000+ hours human validation; confabulation research [4]	Provenance tracking; specialty models; structured eval [15, 16]
Elysian	16x faster insurance claim review	Audit-grade accuracy on 5,400-page claims [8]	Reducto 3-stage ingestion; agentic OCR with confidence gates [19]

Declarative Pipelines

The patterns above—operators, validation, routing, decomposition—converge in **declarative LLM pipelines**: systems where you specify *what* to do (in config) and the platform handles *how* (parallelism, batching, retries, optimization).

What you declare:

- Operator type (map, filter, classify, reduce)
- Prompt template
- Output schema
- Model choice
- Grouping keys (for reduce)

What the platform handles:

- Parallelism and batching
- Retries and error handling
- Schema validation between steps
- Checkpoint / resume
- Optimization (decomposition, gleaning)

Every step declares an `output_schema` or `labels`—structured output is the contract between pipeline steps.

A Declarative Pipeline in YAML

Pipeline config

```yaml
pipeline:
  - name: extract_fields
    type: map
    prompt: prompts/extract_contract_fields.txt
    output_schema: ContractFields
    model: gpt-4o-mini

  - name: validate_extraction
    type: filter
    condition: "output.confidence > 0.8"

  - name: classify_risk
    type: classify
    prompt: prompts/classify_risk_level.txt
    labels: [low, medium, high, critical]
    model: gpt-4o

  - name: summarize_by_vendor
    type: reduce
    group_by: vendor_name
    prompt: prompts/summarize_vendor_contracts.txt
    model: gpt-4o
```

The platform can validate outputs against the schema automatically and route parse failures to retry logic.

Why Not a DAG?

Early pipeline systems modeled workflows as directed acyclic graphs (DAGs)—step A feeds step B feeds step C, no cycles. This works for simple ETL but breaks for LLM pipelines because:

- **Gleaning needs loops:** generate → validate → re-prompt → validate again
- **Confidence routing needs branching:** high-confidence records skip expensive steps; low-confidence records get extra processing
- **Human review needs pause/resume:** the pipeline suspends on uncertain records and resumes when a human provides input

Real LLM data pipelines need **event-driven** or **state-machine** orchestration, not rigid DAGs. The pipeline config may look linear, but the runtime may loop, branch, and checkpoint underneath.

Unit 10 bridge: Staged handoffs, shared context, and synchronization are richer than a fixed DAG—even when the YAML looks linear.

Reusable Operators via MCP

When an operator is valuable across multiple pipelines or clients, package it as an MCP server (from Unit 5):

Contract extraction MCP server

```
from mcp.server import Server
from mcp.types import TextContent

app = Server("contract-extraction")

@app.tool()
async def extract_contract_fields(
    document_text: str,
) -> list[TextContent]:
    """Extract vendor, dates, value, and risk from a contract."""
    result = await call_llm(
        prompt=EXTRACT_PROMPT.format(text=document_text),
        output_schema=ContractFields,
        model="gpt-4o-mini",
    )
    return [TextContent(
        type="text",
        text=result.model_dump_json(),
    )]
```

Now any client—batch pipeline, Slack bot, Cursor workflow, analyst's notebook—calls `extract_contract_fields` without reimplementing. The batch pipeline calls it 50,000 times; the Slack bot calls it once. Same server, same prompt, same schema.

Pipelines as Skills

Recall **skills** from Unit 5—reusable, file-based workflow expertise that an agent loads on demand. A declarative pipeline config is essentially a **machine-readable skill**:

- It encodes the **procedure** ("extract, then validate, then classify, then summarize")
- It specifies the **tools** (which model, which schema, which prompt)
- It defines the **quality checks**

When Harvey builds a workflow for SPA deal-point extraction, or Walmart builds one for catalog enrichment, they're **codifying expertise** that was previously in a human analyst's head.

The pipeline *is* the skill—portable, versionable, and executable. A new team member reads the skill and understands the pipeline. An agent loads it and executes step-by-step.

SKILL.md: Codified Pipeline Expertise

SKILL.md for insurance claim review

```
---
name: insurance-claim-review
description: Process commercial insurance claims end-to-end
triggers:
  - "review insurance claim"
  - "process claim documents"
---

## Procedure

1. **Ingest** all claim attachments using Reducto parse API
2. **Classify** each document (policy, incident report, medical
   record, correspondence, photo, estimate)
3. **Extract** structured fields per document type
4. **Resolve** entities across documents
5. **Validate** extracted fields against policy terms
6. **Summarize** the full claim with citations to source documents
7. **Flag** any inconsistencies for human adjuster review

## Quality checks
- Every extraction must pass schema validation
- Cross-document entity resolution must link >=90% of providers
- Summary must cite at least one source document per claim
```

DocETL and DocWrangler

DocETL (UC Berkeley EPIC Lab) [1] takes the declarative pattern further with an **optimizer** that can automatically rewrite pipelines for better accuracy:

- Insert decomposition steps when documents are too long
- Add gleaning loops when single-pass accuracy is insufficient
- Detect when a resolve operation should precede a reduce

DocWrangler [18] adds **interactive development**:

- Inspect outputs at each pipeline step
- Annotate errors in the UI
- Have the system **refine prompts** based on your annotations

This is the Analyze–Measure–Improve loop from Unit 3, applied to pipeline development.

Key Takeaways (1–4)

1. **LLMs are data operators, not just chatbots.** The primitives—map, filter, classify, reduce, resolve, enrich—are the building blocks. DoorDash and Instacart use them because LLMs solve the cold-start problem that traditional NLP can't.
2. **Errors compound; validate between steps.** The extract-then-validate pattern (Walmart, Harvey) is universal. Two agents—one to work, one to check—is the simplest reliable architecture.
3. **Route models by difficulty.** Amazon's consensus routing and Hebbia's multi-model orchestration show that not every record needs the same model. Prove with expensive, run with cheap.
4. **Fine-tuning is a cost optimization, not a capability unlock.** Ramp's trajectory: general model $\rightarrow$ measure $\rightarrow$ fine-tune $\rightarrow$ 79% savings. Proof first, then optimize for production economics.

Key Takeaways (5–7)

5. **Eval at scale = invariants + sampling + LLM-judge.** Spotify's profile-aware judge is the gold standard for scaling subjective eval without A/B testing everything.
6. **In high-stakes domains, provenance is mandatory.** Abridge and Harvey both trace every output to its source. If you can't show where it came from, you can't use it.
7. **Ingestion is often the hardest step.** Reducto's three-stage pipeline for 5,400-page insurance claims shows that parsing precedes understanding.

The meta-pattern

Every case study follows the same arc: define the operator → prove accuracy on a sample → solve economics at scale → build eval infrastructure → earn trust through provenance. That's P^3 applied to data processing.

Further Reading: Research Papers

1. Shankar et al., *DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing* (2024)—`arxiv.org/abs/2410.12189`
2. Zhu et al., *Transforming Podcast Preview Generation: From Expert Models to LLM-Based Systems*, ACL 2025 Industry Track—`aclanthology.org/2025.acl-industry.26`
3. Fabbri et al., *Evaluating Podcast Recommendations with Profile-Aware LLM-as-a-Judge*, RecSys 2025—`arxiv.org/abs/2508.08777`
4. Liang et al., *The Science of Confabulation Elimination: Toward Hallucination-Free AI-Generated Clinical Notes*, Abridge (2025)—`abridge.com/ai/science-confabulation-hallucination-elimination`

Further Reading: Engineering Blogs (1/2)

5. Harvey AI, *BigLaw Bench Workflows: SPA Deal Points* (Sep 2024)—`harvey.ai/blog/biglaw-bench-workflows-spa-deal-points`
6. Harvey AI, *Introducing BigLaw Bench* (Sep 2024)—`harvey.ai/blog/introducing-biglaw-bench`
7. OpenAI, *Hebbia's Deep Research Automates 90% of Finance and Legal Work* (2025)—`openai.com/index/hebbia`
8. Reducto, *How Elysian Uses Reducto to Review Insurance Claims 16x Faster* (Sep 2025)—`reducto.ai/blog/reducto-elysian-case-study`
9. Walmart Global Tech, *How Walmart Uses LLMs to Manage Its Massive Product Catalogs* (May 2025)—`tech.walmart.com/.../using-llms-to-manage-product-catalogs`
10. AWS ML Blog, *How the Amazon.com Catalog Team Built Self-Learning Generative AI at Scale with Amazon Bedrock* (2025)

Further Reading: Engineering Blogs (2/2)

11. Instacart Tech, *Scaling Catalog Attribute Extraction with Multi-modal LLMs* (Aug 2025)—`instacart.com/.../scaling-catalog-attribute-extraction...`
12. Instacart Tech, *Simplifying Large-Scale LLM Processing Across Instacart with Maple* (Aug 2025)—`instacart.com/.../simplifying-large-scale-llm-processing...maple`
13. DoorDash Engineering, *Building DoorDash's Product Knowledge Graph with Large Language Models* (2024)
14. Modal, *How Ramp Automated Receipt Processing with Fine-Tuned LLMs* (2025)—`modal.com/blog/ramp-case-study`
15. Abridge, *Innovation at the Speed of Trust: Continuous Improvement in Clinical AI* (Dec 2025)—`abridge.com/blog/clinical-ai-specialty-models`
16. Abridge, *Pioneering the Science of AI Evaluation*—`abridge.com/ai/science-ai-evaluation`

Further Reading: Tools and Platforms

17. DocETL documentation—`docetl.org`
18. DocWrangler blog post—*Interactive LLM-Powered Data Processing with DocWrangler* (2025)—`data-people-group.github.io/blogs/2025/01/13/docwrangler/`
19. Reducto technical overview—*Hybrid Architecture: Agentic OCR Deep Dive*—`llms.reducto.ai/hybrid-architecture-agentic-ocr-deep-dive`

Thank you!